Phase 2 Report — Dataset Preparation and Preprocessing

Project: AI-Based Cloud Detection and Localization in Satellite Images

Course: Software Project Management (SPM)
Phase: Phase 2 — Dataset Preparation and Preprocessing
Team: Fatima Fahad, Ayesha Mushtaq, Tayyaba, Samra Syed, Meerab Shafiq

---

1. Phase Overview

Phase 2 focused on preparing the selected satellite-image dataset for subsequent AI model development.

According to the project charter, Phase 2 is specifically defined as “Dataset preparation and preprocessing”, following Phase 1 requirements, literature review, and dataset selection.

The objective of this phase was to obtain a suitable labeled dataset, verify its structure and labels, create a leakage-safe train/validation/test split, and develop a preprocessing/data-loading pipeline that can provide correctly formatted inputs to the U-Net segmentation model.

The prepared dataset and preprocessing pipeline form the first major technical deliverable of the project.

---

2. Dataset Selection

The team selected the 38-Cloud dataset, a labeled satellite-image dataset designed for cloud segmentation.

The dataset was selected because it provides:

- Satellite imagery suitable for cloud detection
- Corresponding cloud masks/labels

- Multiple spectral bands
- Data suitable for a segmentation-based approach
- A sufficiently structured dataset for developing and evaluating a U-Net model

The project requires the system to detect cloud regions and, where supported by the model, highlight/localize those regions.

---

3. Dataset Acquisition

The 38-Cloud dataset was downloaded and prepared in Google Colab.

The dataset was extracted into the Colab working environment and checked to ensure that the required satellite-image bands, masks, and metadata were accessible.

The project uses Google Colab as the development environment, consistent with the technology stack specified in the project charter.

---

4. Dataset Structure and Input Format

The preprocessing pipeline works with four satellite-image bands:

- Red
- Green
- Blue
- Near-Infrared (NIR)

These four bands are loaded and combined into a single input tensor.

The resulting model input has the shape:

384 × 384 × 4

The corresponding cloud mask has the shape:

384 × 384 × 1

The mask is converted to binary values:

0 → non-cloud
1 → cloud

This format is compatible with the planned U-Net segmentation architecture.

---

5. Label Verification

Before using the dataset for model development, sample image/mask pairs were visually inspected.

The purpose of this step was to verify that:

- The cloud masks correspond to the visible cloud regions.
- Masks are aligned with their corresponding satellite images.
- The dataset does not contain obvious labeling or alignment problems.

This step is important because poor dataset quality was identified as one of the project's risks, with the mitigation being the use of a suitable labeled dataset and verification of samples.

---

6. Scene-Level Dataset Splitting

A major part of Phase 2 was creating a scene-level train/validation/test split.

Instead of randomly splitting individual image patches, patches belonging to the same satellite scene were kept together.

The resulting groups were:

Training set
↓
Validation set
↓
Official test set

This approach helps prevent patches originating from the same satellite scene from appearing in different splits.

This is particularly important for satellite imagery because randomly splitting neighboring patches can result in data leakage and overly optimistic evaluation results.

---

7. Leakage Verification

After creating the splits, a dedicated leakage check was performed.

The split was not handed off until the leakage check passed.

The pipeline therefore verifies that the scene-level grouping has been maintained before the split artifacts are saved.

The final result was:

OVERALL LEAKAGE CHECK: PASS

This means the prepared train/validation/test split passed the implemented scene-level leakage verification.

---

8. Dataset Split Artifacts

The following files were generated and saved in the project's dataset_split directory:

```
dataset_split/
│
├── train_patches.csv
├── val_patches.csv
├── test_patches.csv
├── split_manifest.json
└── split_report.md
```

train_patches.csv: Contains the patches assigned to the training set.

val_patches.csv: Contains the patches assigned to the validation set.

test_patches.csv: Contains the patches assigned to the test set.

plit_manifest.json: Stores information required for reproducibility, including:

- Dataset information
- Split methodology
- Random seed
- Validation fraction
- Software version
- Creation timestamp
- Scene IDs
- Patch counts
- Leakage-check status

split_report.md: Provides a human-readable record of the split, including scene counts, patch counts, and scene IDs.This supports the project's non-functional requirement that the model and results should be reproducible using the documented dataset and procedure.

---

9. Preprocessing Pipeline

- A separate Python preprocessing/data-pipeline module was developed:

  ->data_pipeline_cloud_segmentation.py

- The pipeline performs the following operations:

CSV split
↓
Locate corresponding TIFF bands
↓
Load 4 spectral bands
↓
Stack bands
↓
Normalize image values
↓
Load corresponding cloud mask
↓
Binarize mask

↓
Create TensorFlow dataset

---

10. Normalization

Per-band normalization statistics were calculated using the training split only.

The pipeline calculated the following approximate values during the successful sanity check:

Band	Mean	Standard Deviation
Band 1	9839.69	10993.70
Band 2	9709.41	10707.90
Band 3	10352.25	11283.79
Band 4	10743.17	11343.54

Using the training split for these statistics prevents validation/test information from influencing preprocessing.

---

11. TensorFlow Data Pipeline Verification

The completed preprocessing pipeline was tested using the actual 38-Cloud data.

A batch was successfully loaded with the following dimensions:

Image batch: (8, 384, 384, 4)

Mask batch: (8, 384, 384, 1)

The mask values in the tested batch were: [0.0, 1.0]

This confirmed that:

- The image files could be loaded successfully.

- All four required bands were being combined.
- Images had the expected spatial dimensions.
- Masks had the expected dimensions.
- Masks were successfully converted into binary values.
- The TensorFlow dataset pipeline was functioning correctly.

The final pipeline test produced:

> Pipeline built successfully. No training performed — this script only verifies the loader works and produces correctly-shaped, normalized batches.

---

12. Connection to U-Net Model

The preprocessing pipeline is now ready to provide data to the project's baseline segmentation model.

The planned architecture uses:

Input
384 × 384 × 4
↓
U-Net Encoder
↓
Bottleneck
↓
U-Net Decoder
↓
Output
384 × 384 × 1

The output represents a pixel-level cloud segmentation mask.

This directly supports the project's objective of developing and training an AI model for cloud detection and subsequently localizing/highlighting cloud regions.

---

13. Phase 2 Deliverables

Deliverable	Status
38-Cloud dataset acquired	✅ Completed
Dataset structure checked	✅ Completed
Sample labels visually verified	✅ Completed
Scene-level splitting implemented	✅ Completed
Leakage check	✅ Passed
Training CSV	✅ Completed
Validation CSV	✅ Completed
Test CSV	✅ Completed
Split manifest	✅ Completed
Split report	✅ Completed
Band normalization	✅ Completed
TensorFlow data pipeline	✅ Completed
Real-data sanity check	✅ Passed
U-Net-ready input format	✅ Confirmed

---

14. Phase 2 Outcome

Phase 2 successfully produced a leakage-checked, scene-level dataset split and a functioning preprocessing pipeline for the 38-Cloud dataset.

The final pipeline successfully converts the prepared dataset into normalized four-channel satellite-image tensors of: 384 × 384 × 4 with corresponding binary cloud masks of:

384 × 384 × 1

Therefore, the dataset and preprocessing stage is ready to be integrated with the U-Net baseline model in Phase 3: Baseline Model Development.

The project charter defines Phase 3 as baseline model development, followed by Phase 4 training/evaluation and Phase 5 localization/visualization.

---

15. Phase 2 → Phase 3 Handoff

At the end of Phase 2, the following components are ready for the next phase:

38-Cloud Dataset
↓
Scene-Level Split
↓
Leakage Verification
↓
train_patches.csv
val_patches.csv
test_patches.csv
↓
Preprocessing Pipeline
↓
Normalized 384×384×4 tensors
↓
Binary 384×384×1 masks
↓
U-NET
↓
Baseline Model Development

Current project position:

Phase 1: Requirements, literature review & dataset selection → ✅
Phase 2: Dataset preparation & preprocessing → ✅ COMPLETED
Phase 3: Baseline model development → NEXT

This is also consistent with the original phase structure in the project charter.
